Bucket Versioning, Replication and Static website Hosting in S3

Class No:6

1.Bucket Versioning in AWS S3

- **Definition:** A feature in Amazon S3 that allows you to keep multiple versions of an object (file) in the same bucket.
- **Purpose:** Protects against accidental overwrites and deletions by maintaining history.
- **Behaviour:** When versioning is enabled, every time you upload or modify an object, S3 creates a new version with a unique version ID.

Purpose of Versioning

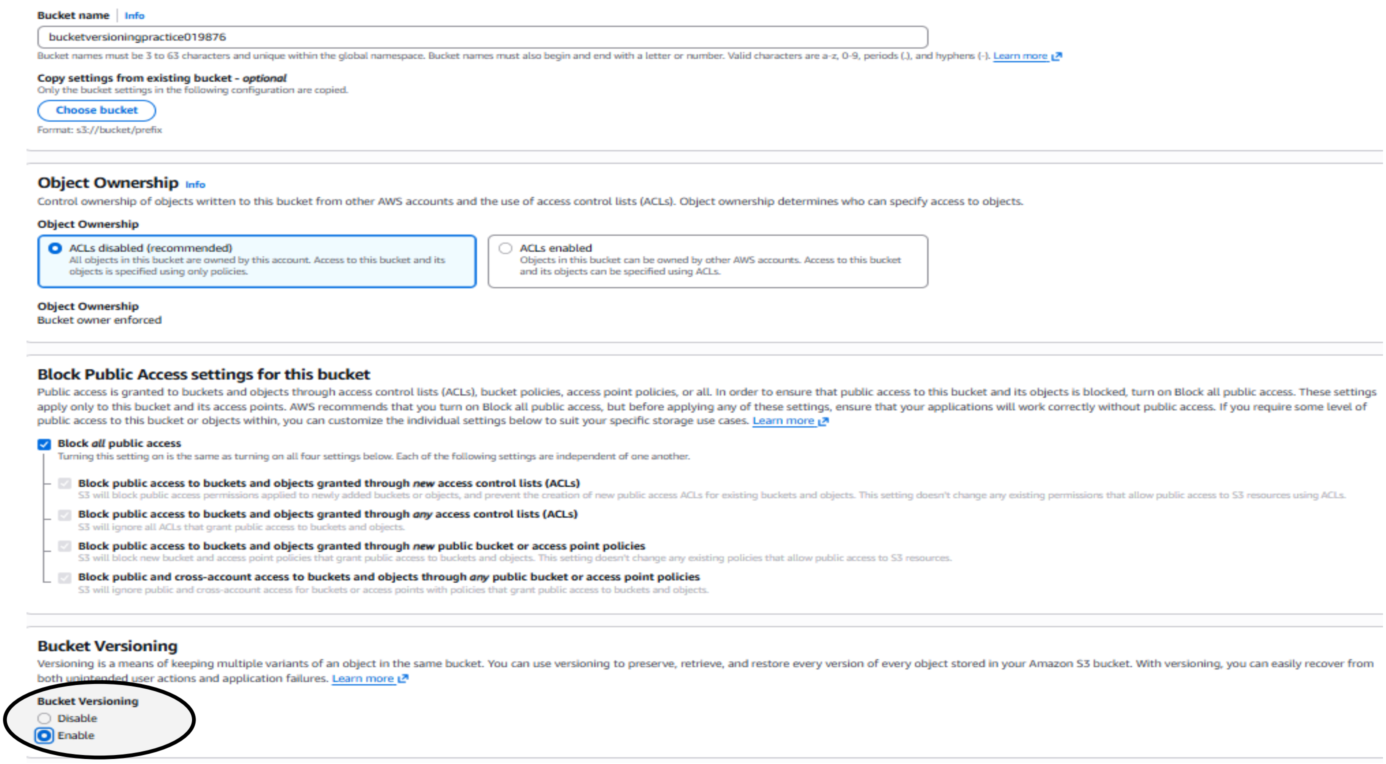
- **Data Protection:** Prevents permanent loss if files are accidentally deleted or overwritten.
- **Recovery:** You can restore older versions of objects.
- **Audit & Tracking:** Maintains history of changes for compliance or debugging.
- **Safe Collaboration:** Multiple users can work on the same bucket without losing data.

Delete Marker

- **Definition:** A special placeholder object created when you delete an object in a versioned bucket.
- **Purpose:** It doesn't remove the object permanently; instead, it makes the latest version appear as "deleted."
- **Behaviour:**
  - When you delete an object, S3 adds a **delete marker** as the newest version.
  - The object is hidden from normal view but older versions are still accessible.
  - Removing the delete marker restores visibility of the object.

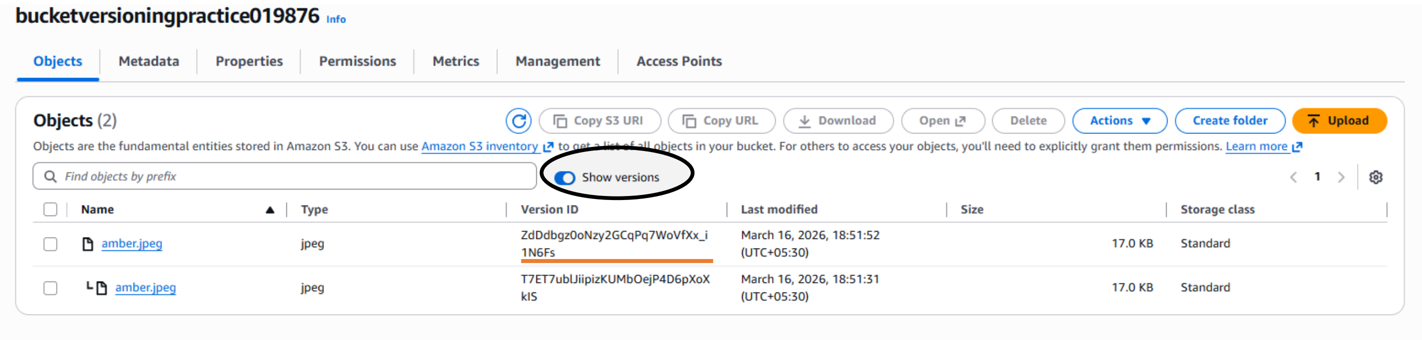
Hands on Bucket Versioning:

Step 1: While creating a S3 bucket**,** Turn on versioning for your S3 bucket.



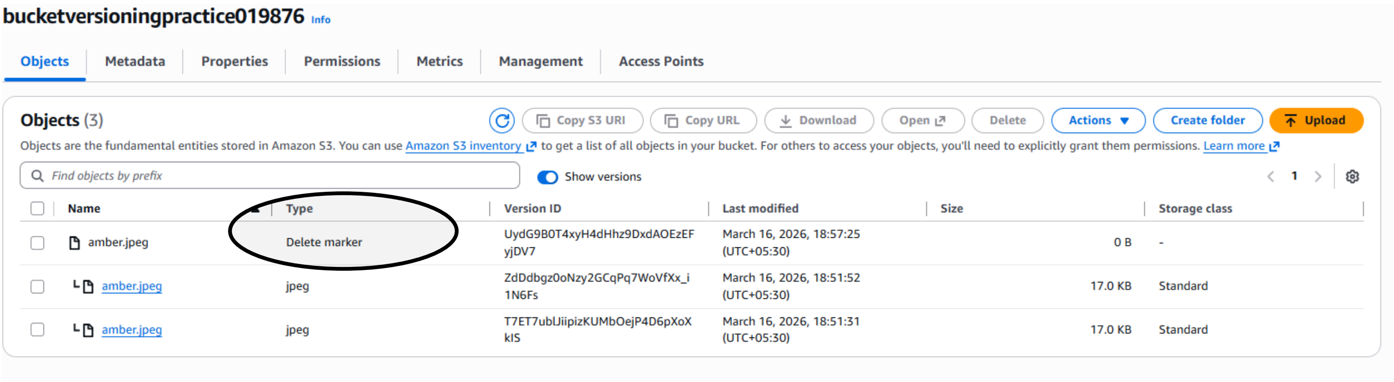
Step 2: Upload Object in the created bucket with same name for two times. Each upload creates a new version with a unique ID. Updates don't overwrite; they create new versions.

(Note: The versions can be seen only the show versions is enabled.)

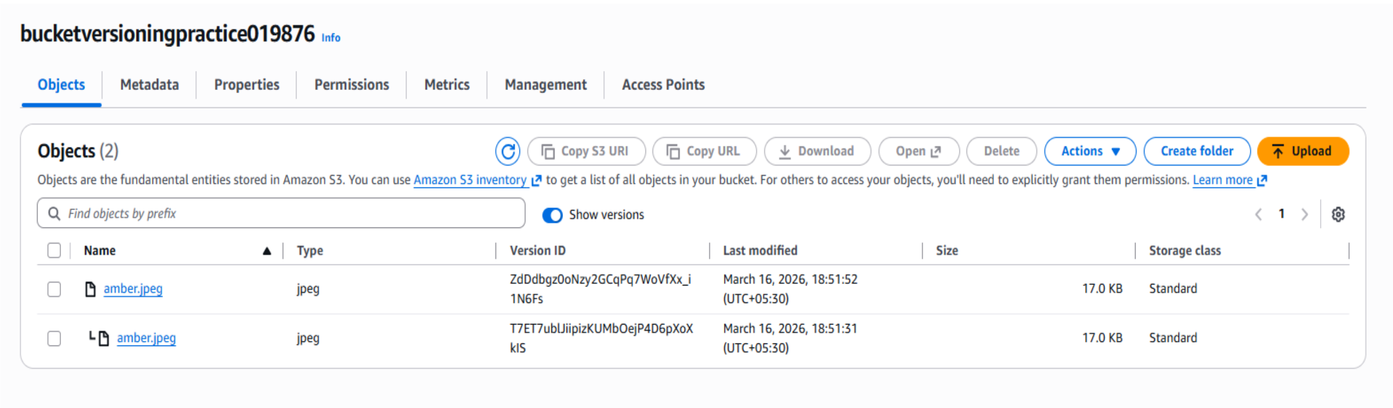


Step 3: Delete Object. Instead of removing data, S3 adds a **delete marker**.

The show versions must be disabled while deleting the objects , then enabled after deletion to see the delete marker.



Step 4: Restore Object by removing the delete marker → the previous version becomes the latest and visible again.



2.Bucket Replication in AWS S3

- **Definition:** Replication is the process of automatically copying objects from one S3 bucket (source) to another (destination).
- **Purpose:** Ensures redundancy, disaster recovery, compliance, and low-latency access by keeping copies of data in different locations.
- **Requirement:** Both source and destination buckets must have **versioning enabled**.

Types of Replication

1. **Cross-Region Replication (CRR)**
    - Copies objects across different AWS regions.
    - Useful for disaster recovery, compliance with data residency laws, and serving global users.
2. **Same-Region Replication (SRR)**
    - Copies objects within the same AWS region.
    - Useful for maintaining copies in different accounts, compliance, or separating workloads.
3. **Cross-Account Replication (CAR)**
    - Copies objects for buckets in two different accounts.

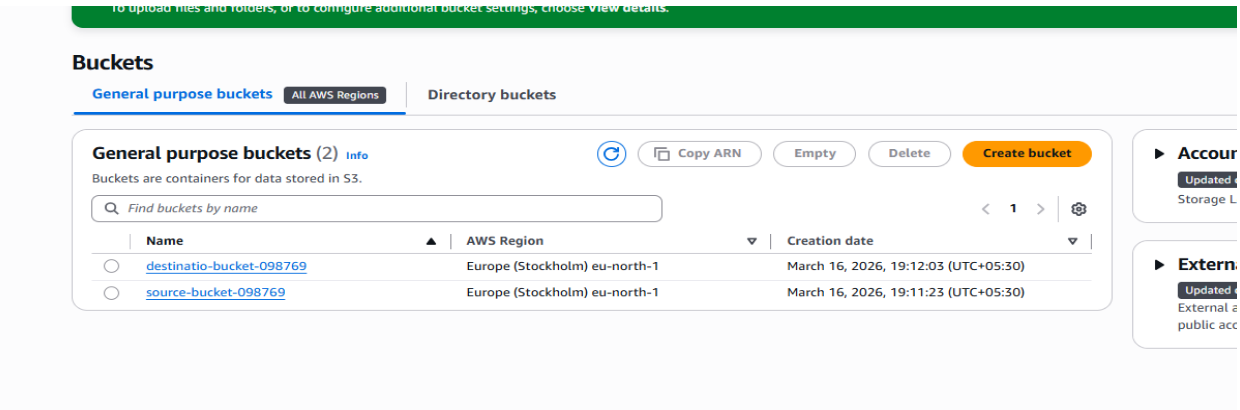
Replication Rules

Replication rules define **what to replicate** and **how**. Key points:

- **Scope:** You can replicate the entire bucket or specific prefixes/tags.
- **Ownership:** Replication can be set up between buckets in the same or different AWS accounts.
- **IAM Role:** A replication IAM role must be created to grant S3 permission to replicate objects.
- **Options:**
    - Replicate existing objects or only new ones. (we used this in our hands-on)
    - Replicate delete markers (optional).
    - Replicate specific metadata (like ACLs, tags).
- **Versioning:** Required on both source and destination buckets.

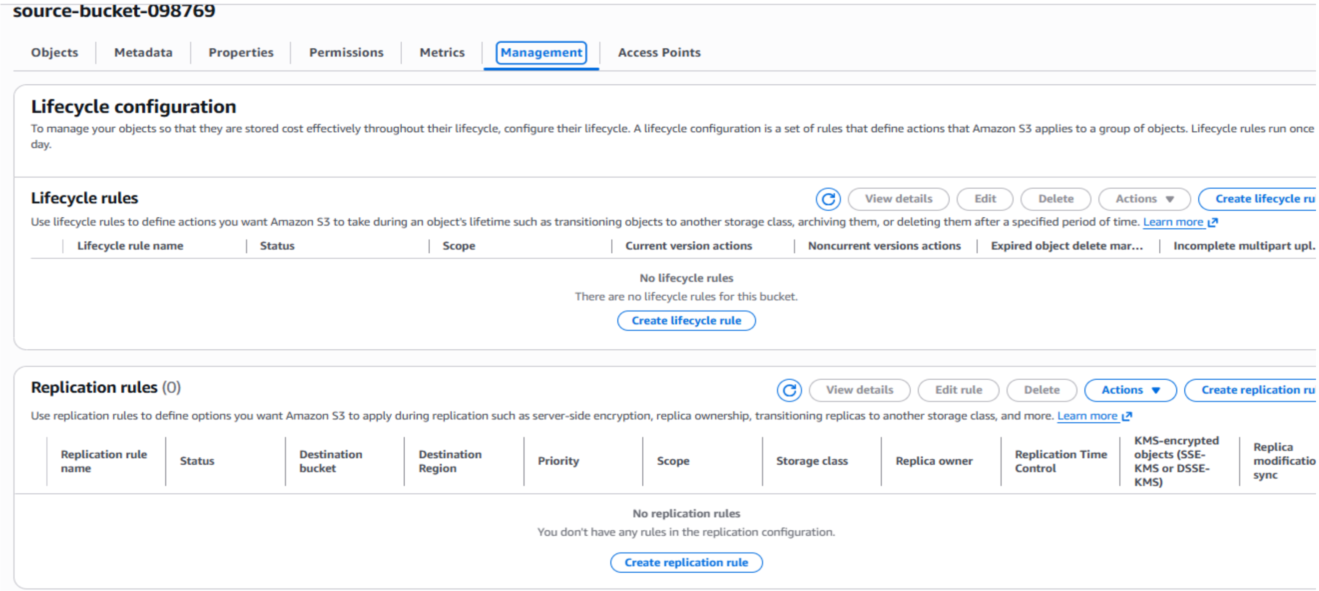
Hands on Bucket replication:

Step 1: Create two Buckets. Source bucket: **source-bucket-098769** and Destination bucket: **destinatio-bucket-098769.** They can be public or private. But, version must be enabled for both.

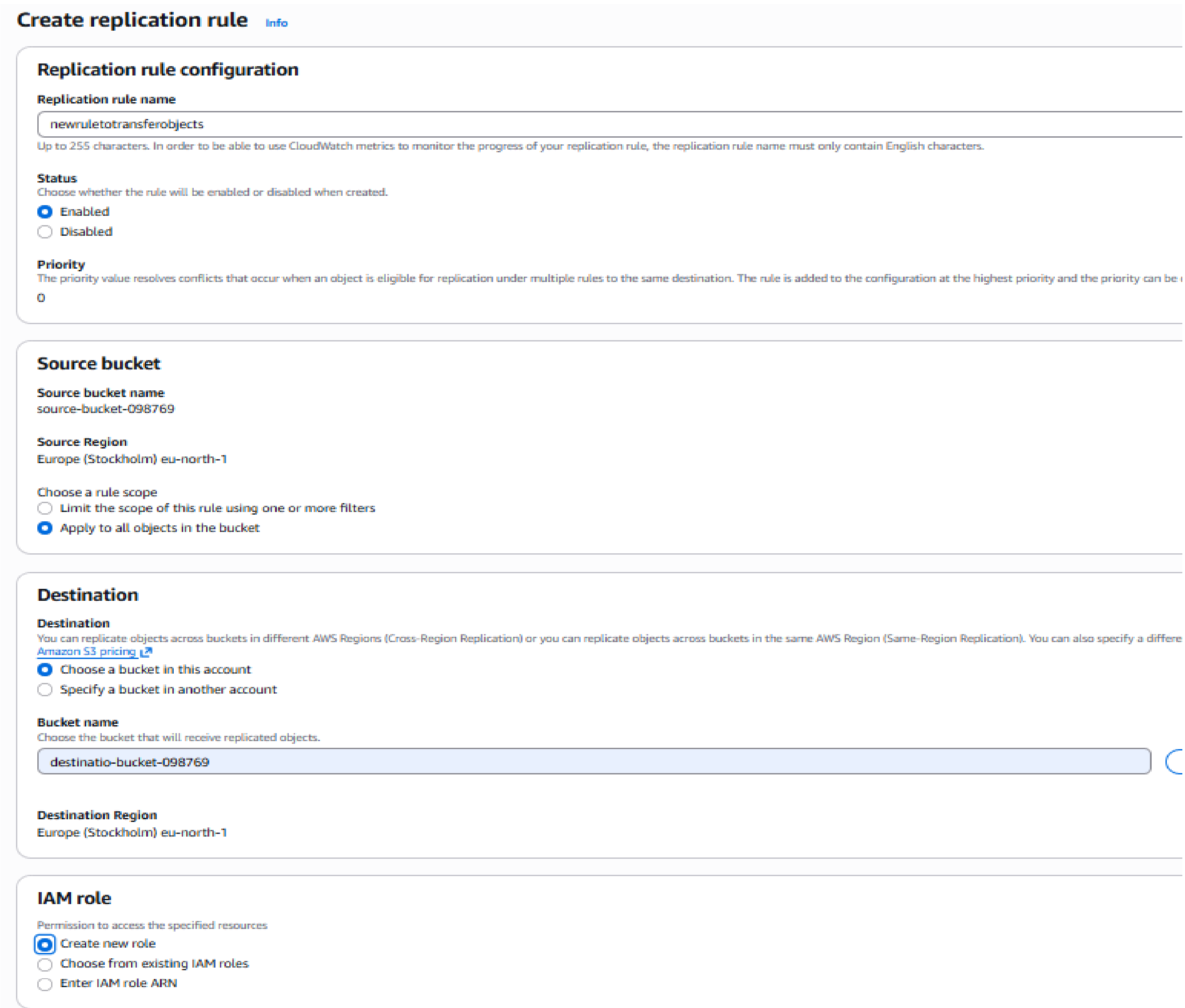


Step 2: Now open the source bucket add some files, click **Management.** Follow the configurations listed below:

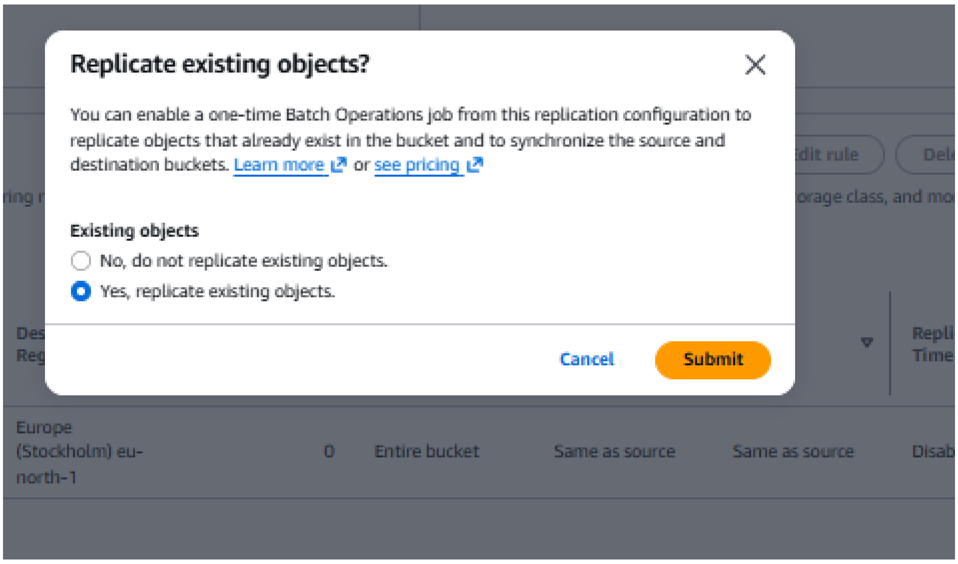
(i) Click the Create Replication rule.



(ii) Provide a rule name: **newruletotransferobjects ,** then enable the status , choose the bucket in this same account. Then search and enter the destination account , choose create new role. Save.

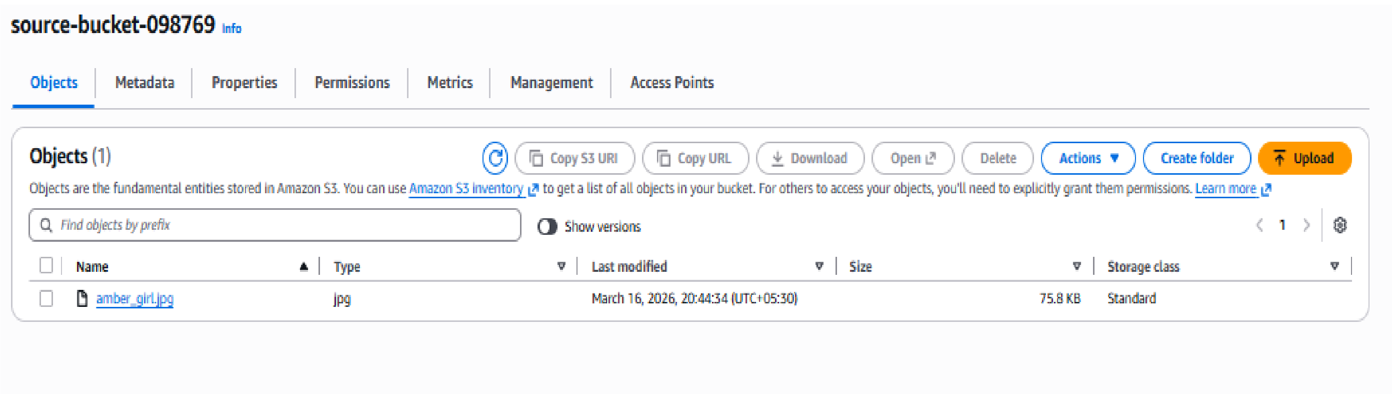


(iii) It will ask for permission. Mark it as yes and submit.

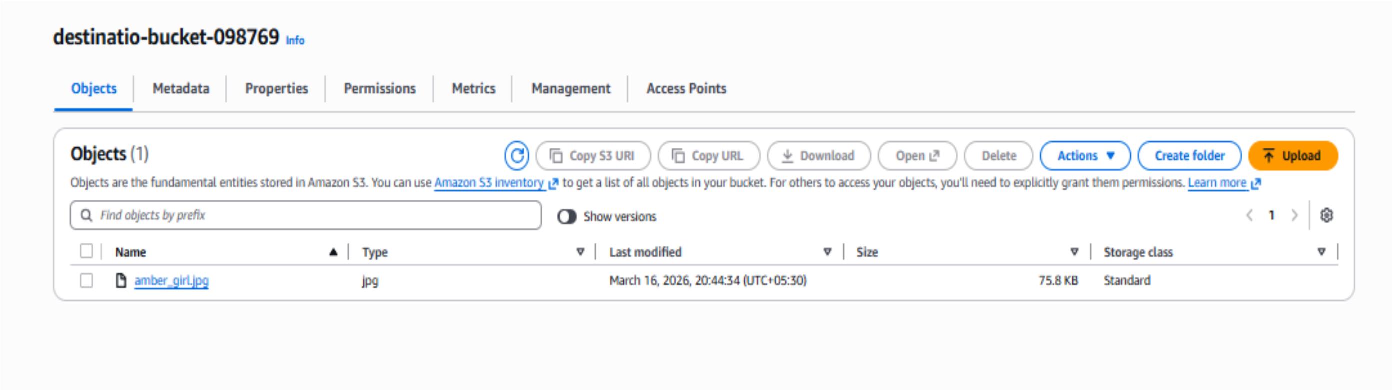


Step 3: Add objects in source bucket and wait for few minutes we can see the object is added automatically in the Destination bucket.

Source bucket:



Destination bucket:



3.Static Web Hosting in S3

- **Definition:** Amazon S3 can serve static content (HTML, CSS, JavaScript, images) directly from a bucket as a website.
- **Purpose:** Host lightweight websites without needing servers, ideal for portfolios, documentation, or landing pages. This is known as serverless. (AWS handles scaling and management)
- **Limitation:** Only supports **static content** — no server-side scripting (like PHP, Node.js).

Purpose of Static Hosting

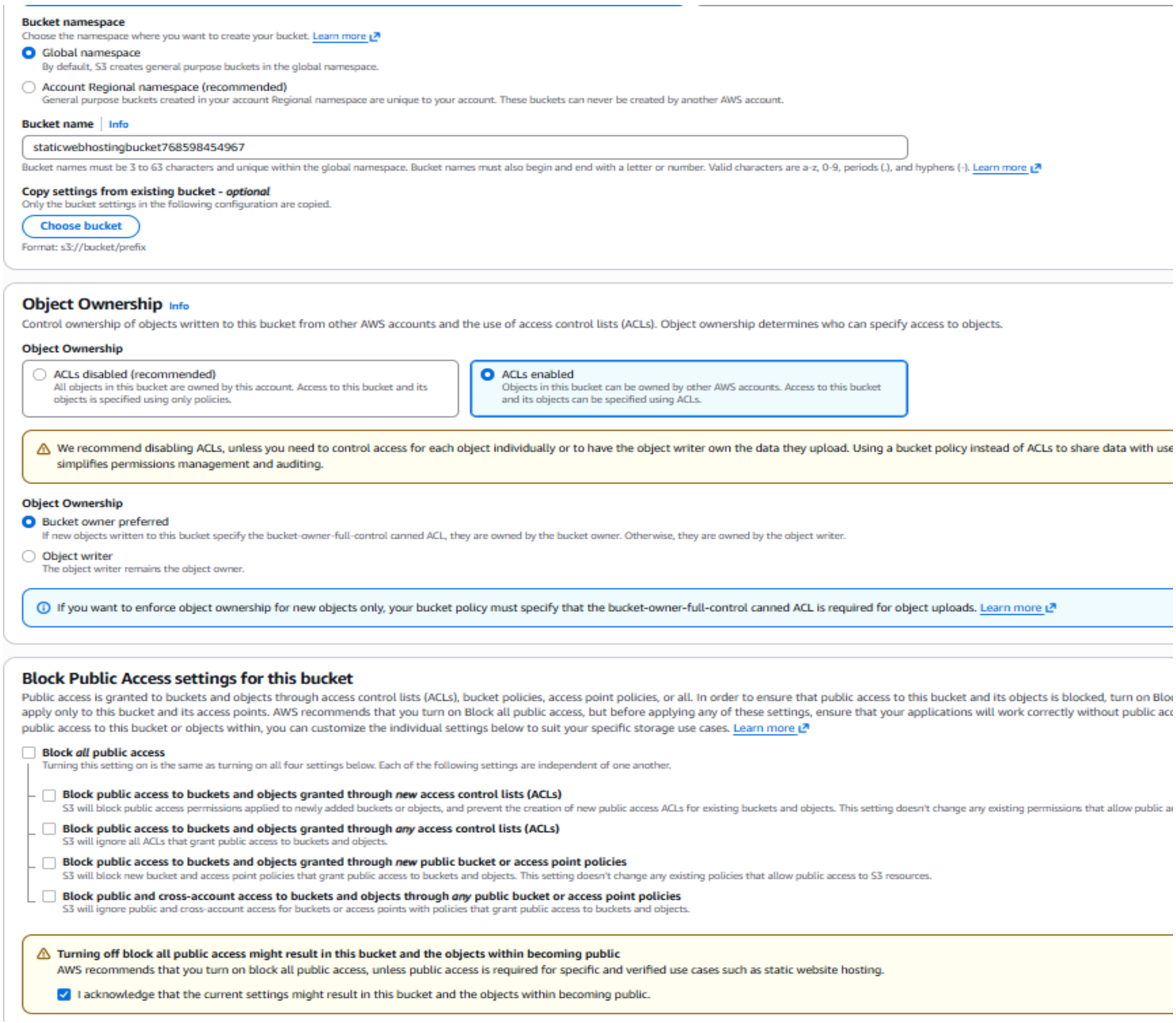
- **Cost-effective:** Pay only for storage and data transfer.
- **Scalable:** Automatically handles large traffic without extra configuration.
- **Simple setup:** No need to manage servers or infrastructure.

Mandatory Things for Static Hosting

- **Bucket must be public** (or objects must be publicly readable) for the website to be accessible.
- **Index document** must be defined (e.g., index.html).

Hands on Steps to Enable Static Hosting

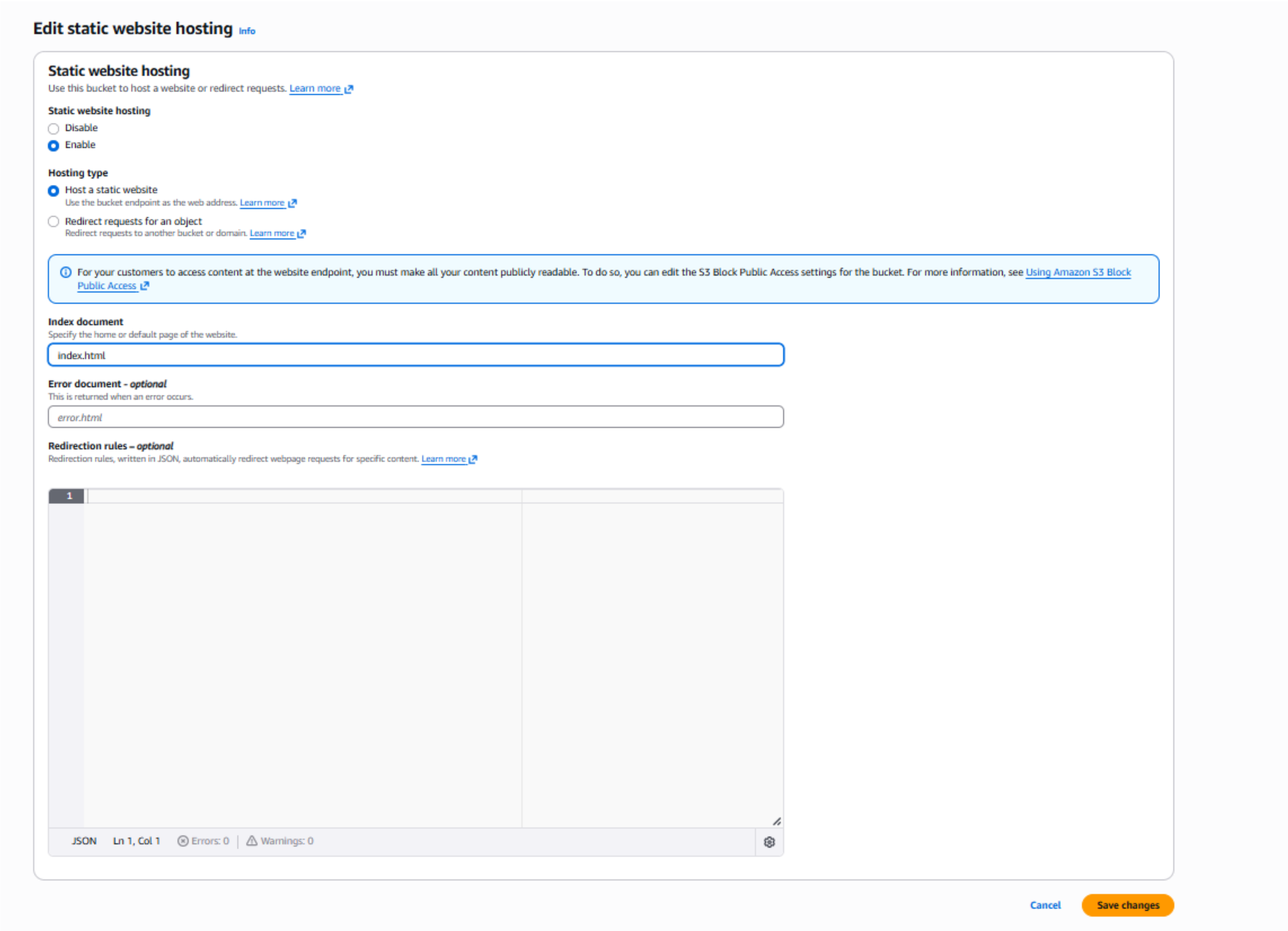
Step 1: Create an S3 Bucket. Name: staticwebhostingbucket768598454967 , make it public by enabling ACL and disable the public access deny option.



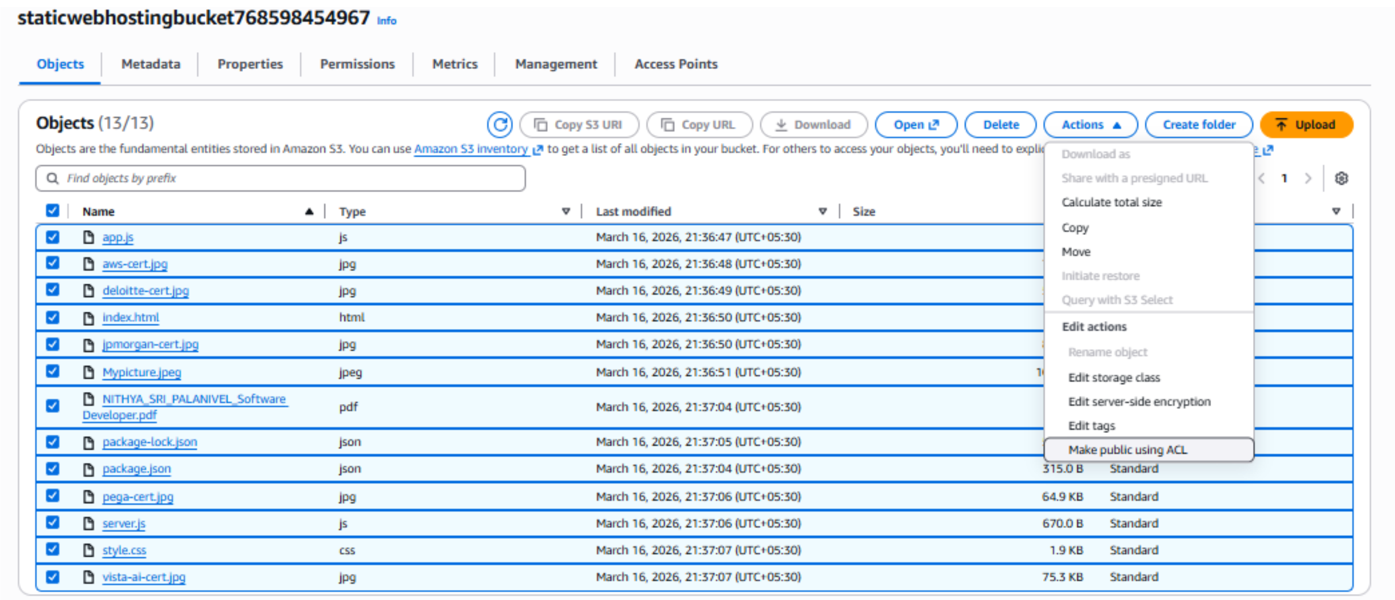
Step 2: Upload Website Files. Upload HTML, CSS, JS, and image files into the bucket. Make sure the path is same, don't change it. The name of the html file should be index.html.

Step 3: Enable Static Website Hosting

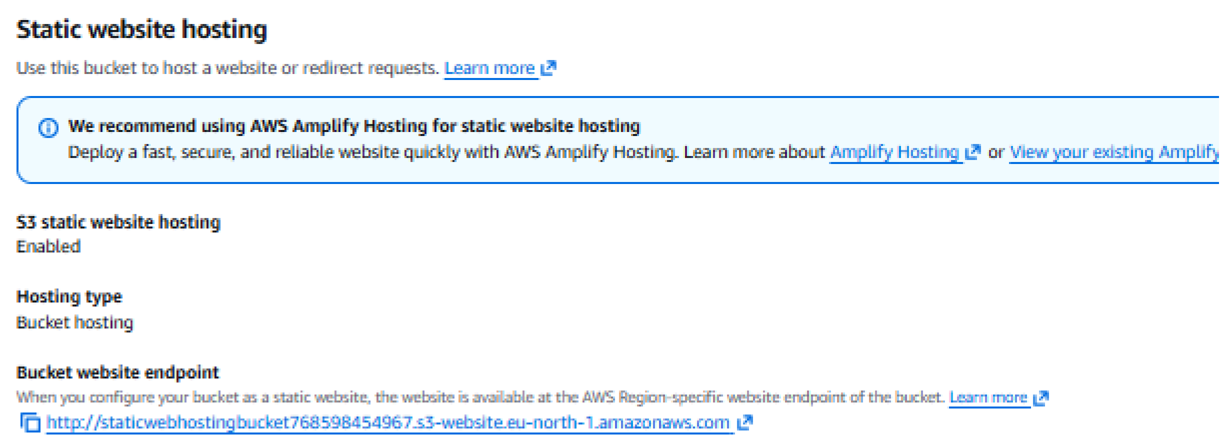
- Go to **Properties** → **Static website hosting** → **Enable**.



Step 4: Set Bucket Policy for Public Access. By default, S3 bucket objects are private. Click all -> go to actions, make all public using ACL enabled.



Step 5: Access Website Endpoint. AWS provides a endpoint under the properties tab of the bucket. (scrool down, it will be available in the static web hosting.)



Step 6: Copy and paste the url in a browser, we can see the contents of the uploaded file.(Here I used my portfolio files.)